

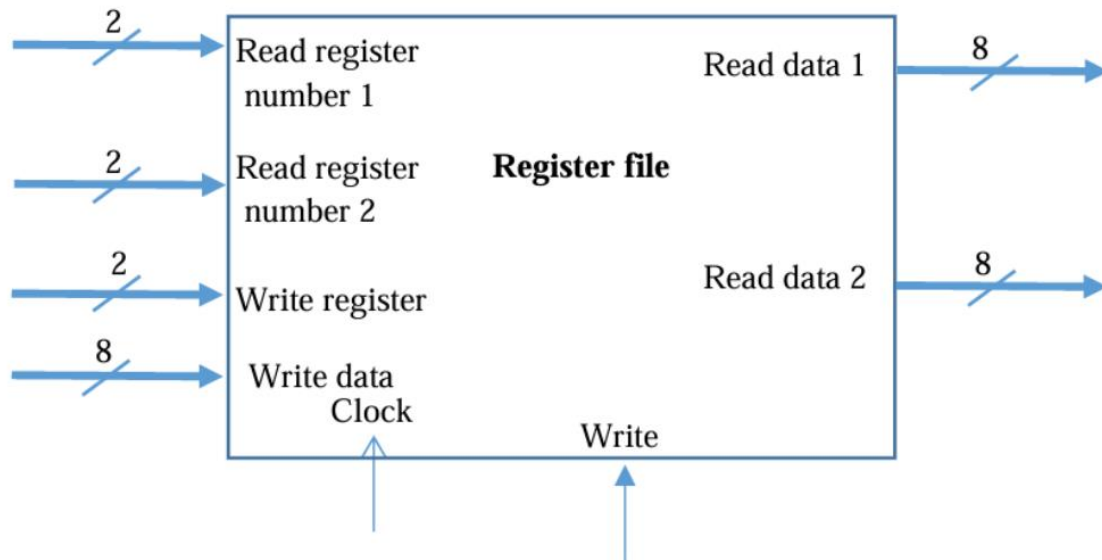
Full name: Doan Cong Thanh

Id: 22125096

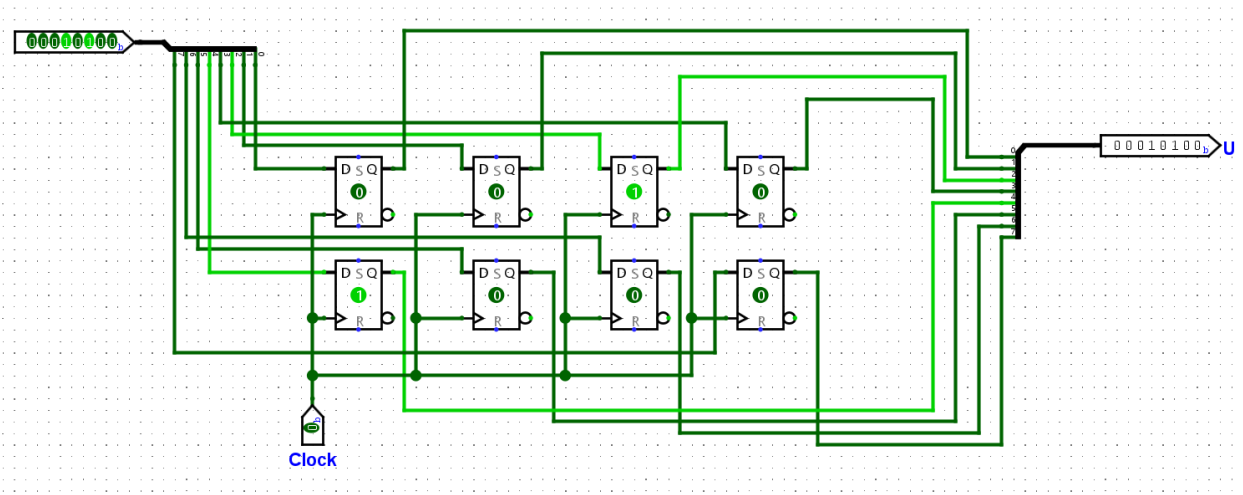
Class: 22TT2

Exercise 01:

We need to design the register file like this:

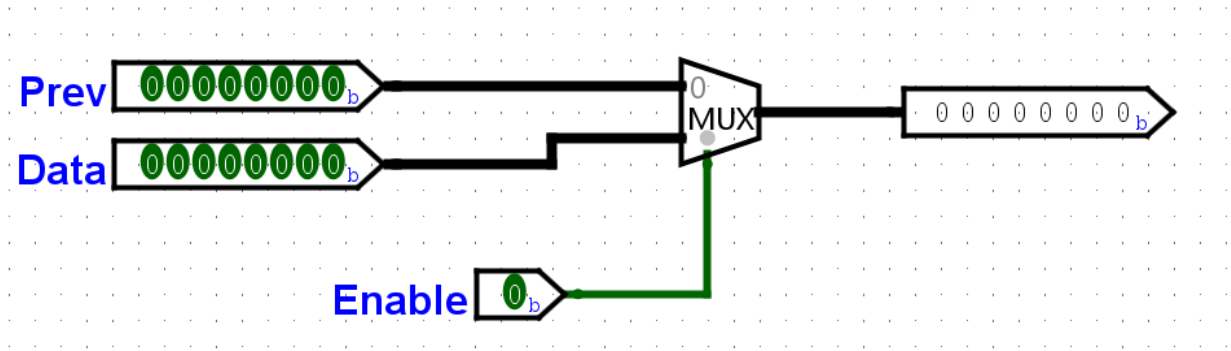


As we can see, the four registers in this Register file obey the D-flip flop mechanism, so we need to build the register first which look like this:



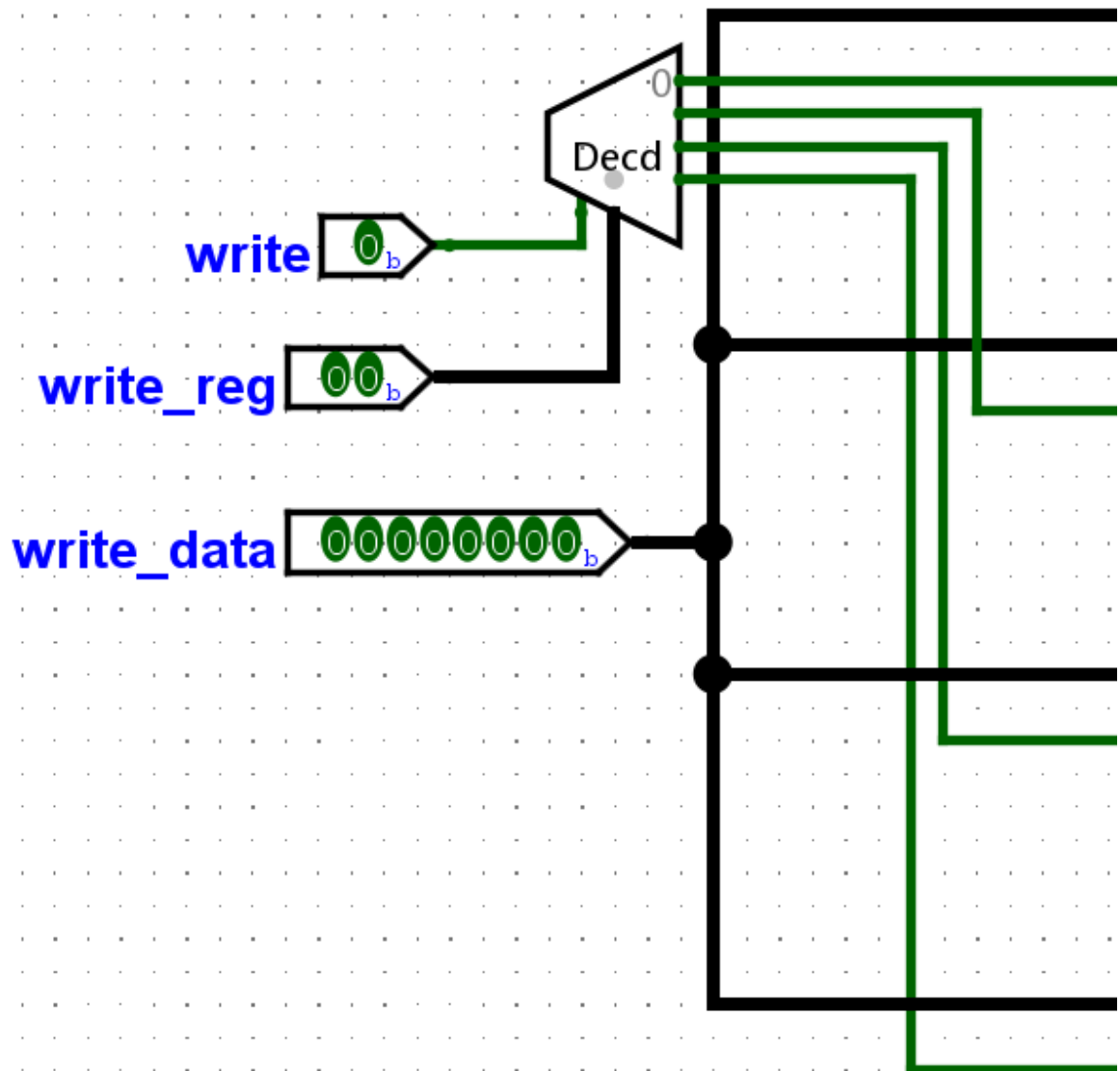
This register has 8-bit data-width.

To design write function for this register, we build the sub-circuit logic first:

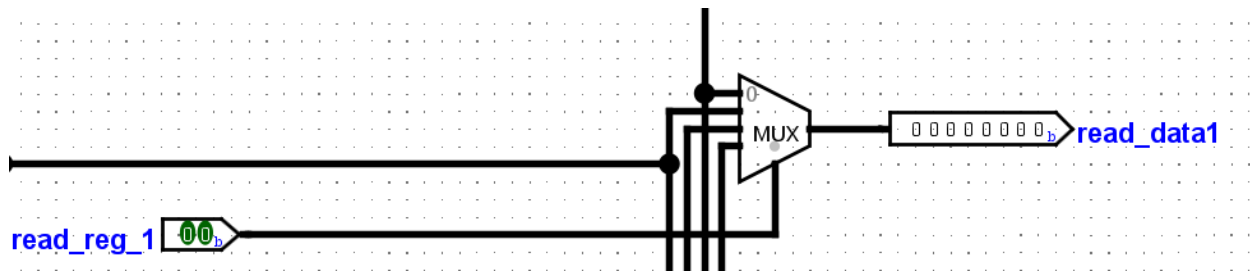


We use multiplexer with 1-bit selection between the previous data and the current one. When we activate the write function (which means turn enable to 1), we will bring the current data (the data in) to the chosen register.

To select among those four registers, we need to build a selection function, which turns out to use decoder:

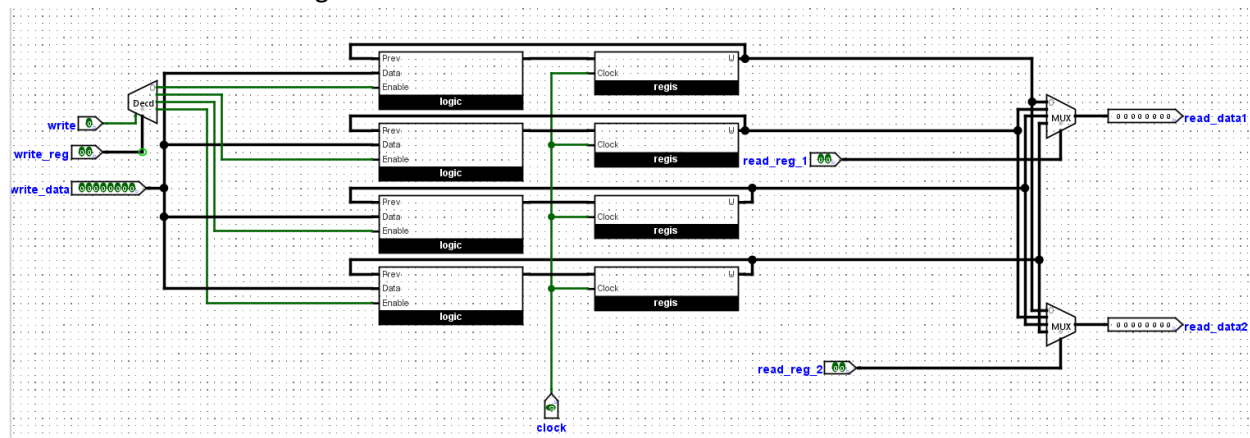


Lastly, the read data operation can be easily achieved by using demultiplexer with 2-bit selection:



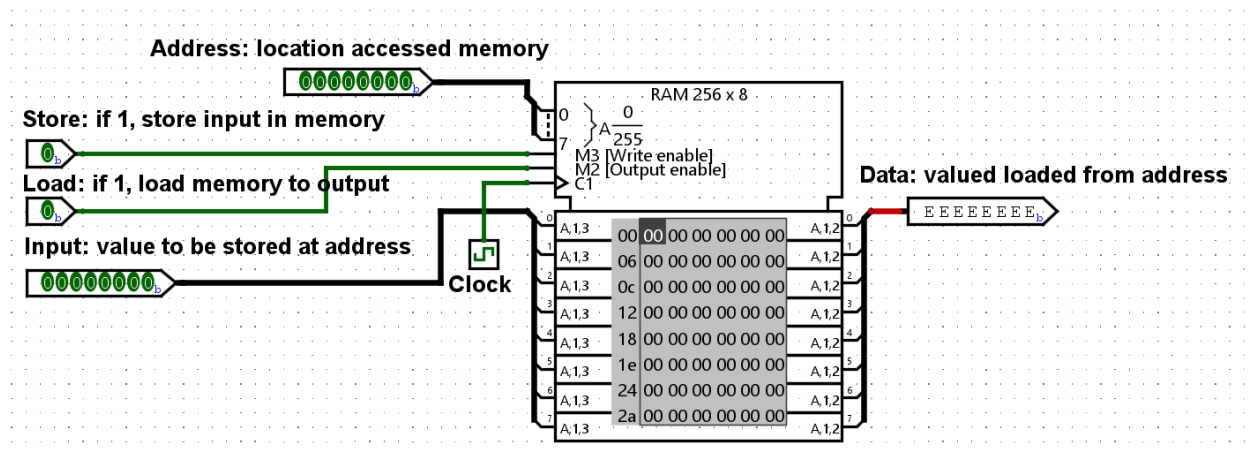
This sub-circuit illustrates how we can select among those data of the four registers. The same process can be seen when we discuss read_reg_2.

The final circuit for this register file is:

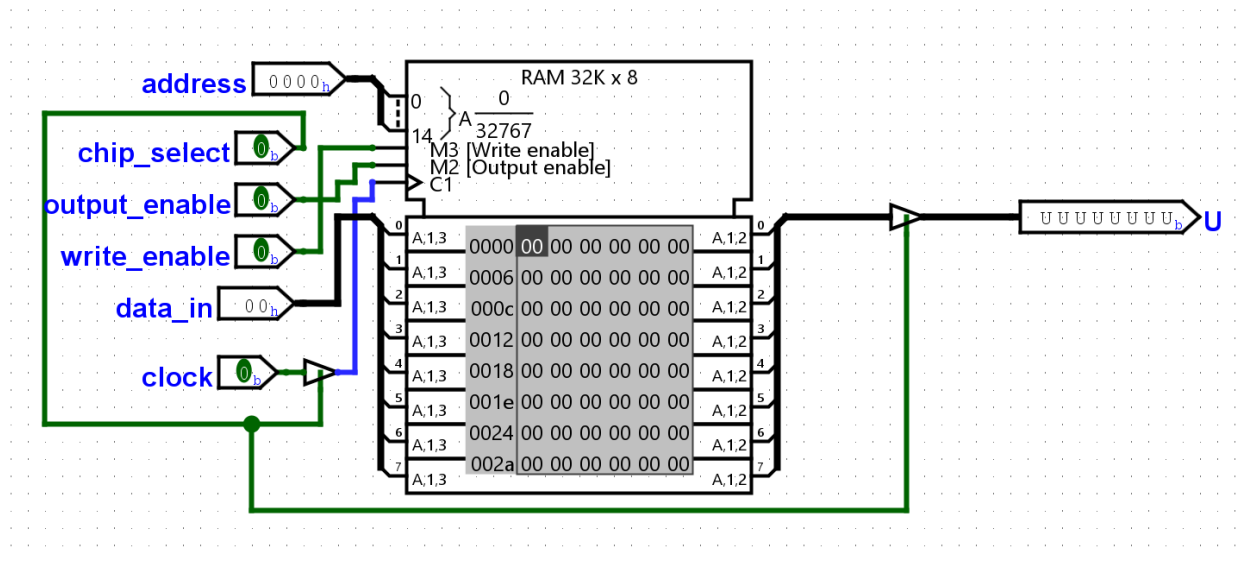


Exercise 02:

Take a quick survey on block RAM in logism:



To build the block RAM 32K x 8 bits, we can use the block RAM in logism, the extra function to choose whether select chip or not, we can use the control buffer and attach it to the clock and the output.



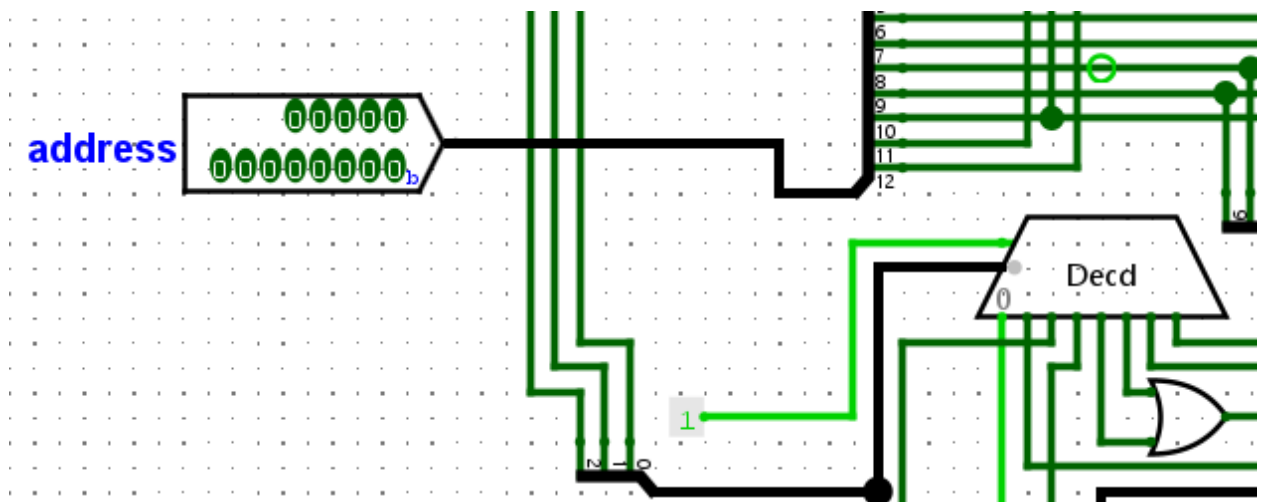
The control buffer works as the control gate wires, when it is active, our wire is connected, otherwise the wire is broken.

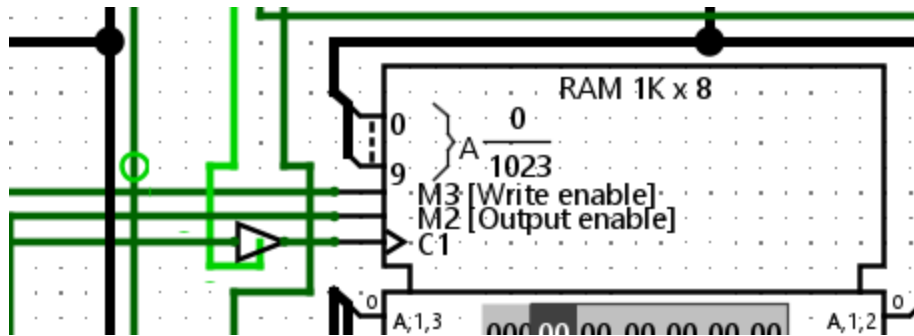
Exercise 03:

In this exercise, I use three 2K RAM and two 1K RAM.

Explain the logic:

- To represent 8K memories, we need to use 13 bits.
- The 2K RAM contains 11 bits and the 1K one contains 10 bits.
- So in this case if we use 13 bits to represent the address, we have 3 bits left and we will use these three bits to select between these RAM block (2K and 1K).
- We can handle this task by using decoder and put the control buffer in front of each RAM block.

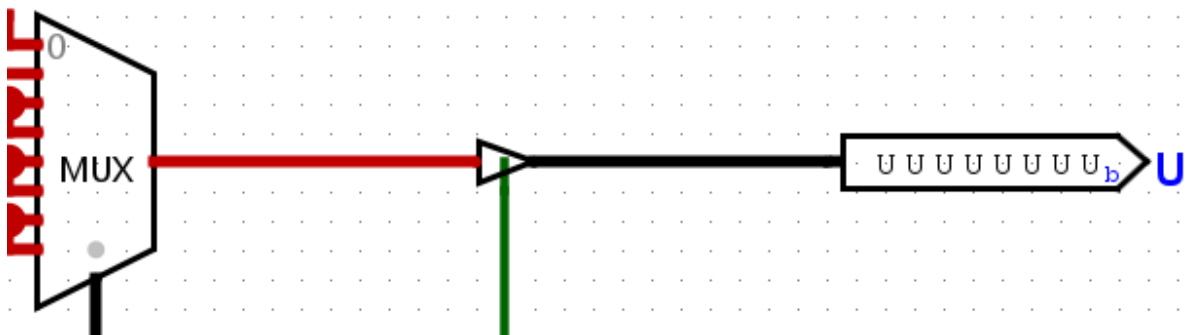




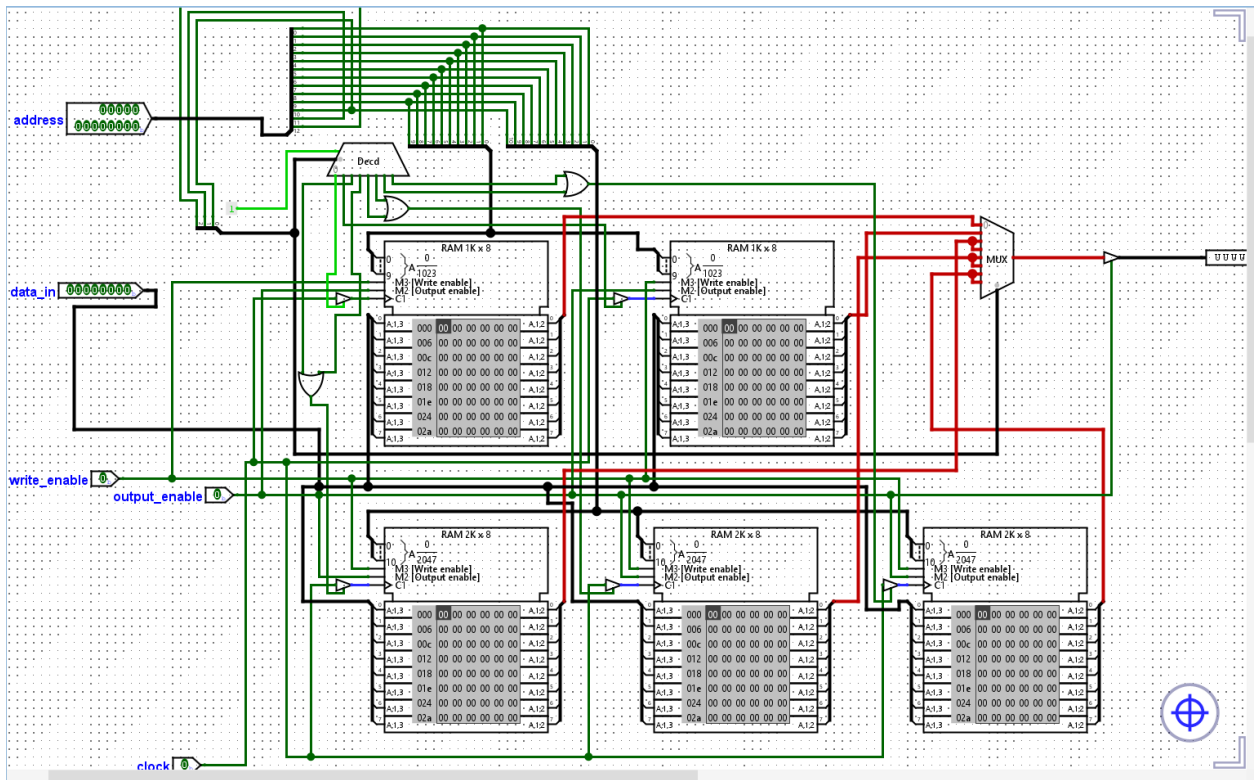
- We have the following table to select among these RAM block:

Selection bits	RAM
000	1K(1)
001	1K(2)
010	2K(1)
011	2K(1)
100	2K(2)
101	2K(2)
110	2K(3)
111	2K(3)

- And for the output, we use the multiplexer similar to the way we apply in exercise 02, with the same 3 bits selection in previous decoder and a control buffer to control the output:

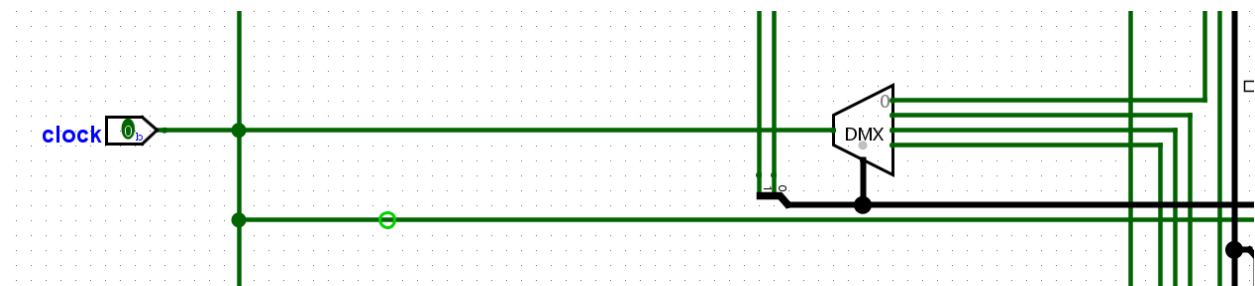


- Lastly, the data input for these 5 RAM block is the same. The difference is the address input, we only use the first 10 bits for the 1K RAM's address and the first 11 ones for the 2K RAM's address
- We have the final circuit for this exercise:

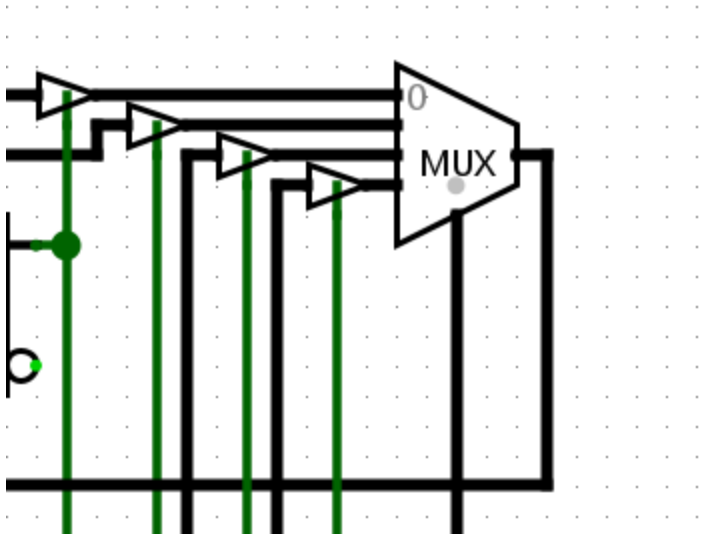


Exercise 04:

The only thing we focus most in this exercise is the bidirectional busses, which means the input and output uses the same gate. And to handle this transfer process, we will take the advantage of the control buffer: when we only enable write data, we then activate the control buffer for the input gate and turn off the control buffer for the output gate, the similar process can be achieved in the opposite way. To build the 32K RAM from 8K RAM, we apply the same approach in exercise 03. But in this time, I do not use the control buffer for each block RAM but use the clock as the input for demultiplexer, as we can see:



Handling output operation using multiplexer which is quite same as the one in exercise 03, the only difference is installing control buffer in front of each output:



- There is still one case we need to handle, when `write_enable = 1` and `output_enable = 1`. For this situation, I will write the data in RAM first, then load the data via the accesses memory.
- This task can be achieved by using this logic: “when `write_enable = 1` and `output_enable = 1` and the change in `data_in` or `address` is detected, then we turn `write_enable = 1` and `output_enable = 0`, then turn `write_enable = 0` and `output_enable = 1`). I will illustrate this logic in the following truth table:

<code>write_enable</code>	<code>out_enable</code>	<code>change</code>	<code>write_next</code>	<code>out_next</code>
0	0	0	0	0
0	0	1	0	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	0	1
1	1	1	1	0

K-map (`write_next`):

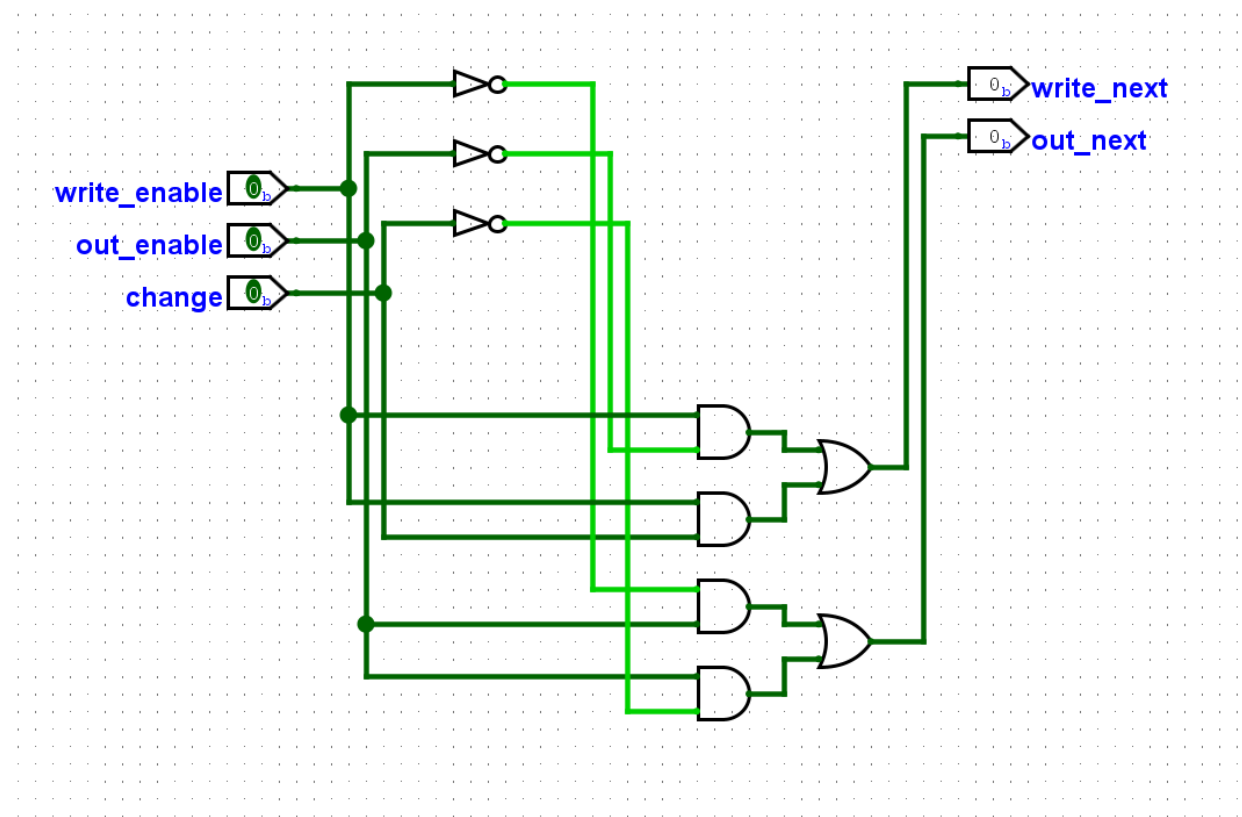
		out_enable, change			
		00	01	11	10
write_enable	0	0	0	0	0
	1	1	1	1	0

$write_next = write_enable \cdot \sim out_enable + write_enable \cdot change$

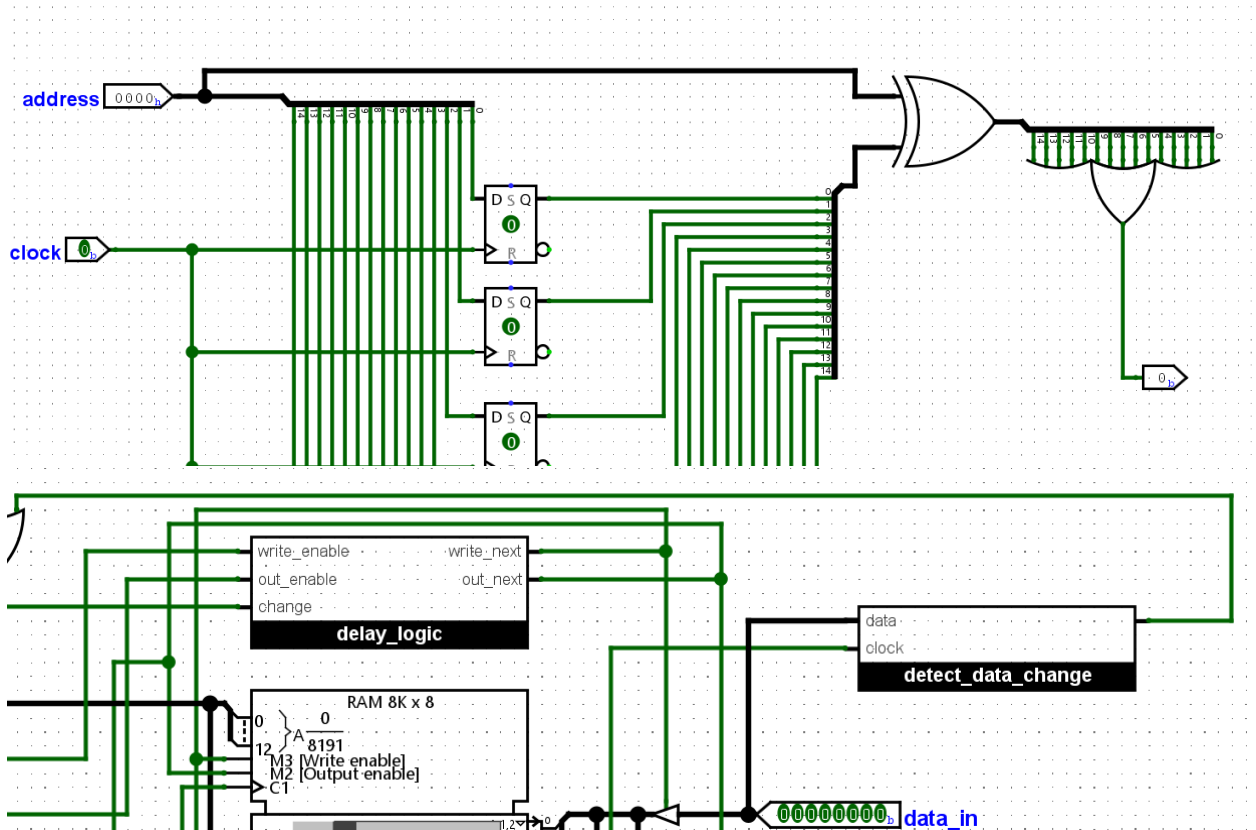
K-map (out_next):

		out_enable, change			
		00	01	11	10
write_enable	0	0	0	1	1
	1	0	0	0	1

$out_next = \sim write_enable \cdot out_enable + out_enable \cdot \sim change$



- When we want to detect whether the data or address changes or not. We use this logic: “use D-flip flop to build the register to store the previous address, as we do in exercise 01. Then XOR the address_in and previous address, we last or each bit together to detect whether the address changes or not”. The same mechanism can be seen when we apply to detect whether the data changes or not



- The final circuit for this problem is:

